Computational Statistics and Simulation: Section 1. Generation of random variables

Master in Mathematics and Applications

Isabel Pereira,
isabel.pereira@ua.pt

Universidade de Aveiro,
2025/26

Section 1: Generation of random variables

1. Motivation
2. Introduction
3. Inverse transform methods
   - Univariate continuous distributions
   - Univariate discrete distributions
   - Multivariate distributions
4. Rejection method
5. Sums and mixtures of random variables
6. Adequacy of the statistical model: Graphical method

0. Motivation

- Calculating the distribution of an estimator (or distribution of a random variable in general)

  Example: $U_1, U_2, \ldots, U_n$ are independent random variables with uniform distribution, $U_{[0;1]}$. Which distribution has $Y = \sum_{i=1}^{12} U_i - 6$?

  n = 12 is large enough to apply the central limit theorem?

- Numerical integration: $\theta = \int_0^1 g(x)dx = ?$

  If $U \sim U_{[0;1]}$, then $\theta = E(g(U)) = \int_0^1 g(u)f(u)du = \int_0^1 g(u)du$

  θ can be estimated by generating a large number of $U_k \sim U_{[0;1]}$ and compute the sample average of $g(u_k)$: $\hat{\theta} = \frac{1}{n}\sum_{k=1}^{n} g(u_k)$

0. Motivation

- Estimating π $\quad \pi = \iint_{\{(x,y):x^2+y^2\le 1\}} dxdy$

Suppose that the random vector (X,Y) is uniformly distributed in the square [-1;1]×[-1;1] of area 4 centered at the origin.

$$P((X,Y)\in \text{circle}) = P\left(X^2+Y^2\le 1\right) = \frac{\text{area of the circle}}{\text{area of the square}} = \frac{\pi}{4}$$

Considering $S = S(X,Y) = \begin{cases} 1 & \text{, if } X^2+Y^2\le 1 \\ 0 & \text{, otherwise} \end{cases}$

Then $E(S) = P\left(X^2+Y^2\le 1\right) = \frac{\pi}{4}$

If we generate a large number of random points in the square, the proportion of points that fall within the circle will be approximately $\pi/4$.

$$\pi \approx 4\times \frac{\sum_{k=1}^{n} s(x,y)}{n}$$

1. Introduction

In this section we will address the subject of how to generate using computer random samples with a given distribution. Since computer programs are inherently deterministic, care must be taken with the way in which numbers are generated randomly in computer.

In general this task is made in 2 steps

- generate observations $U_1, U_2, \ldots$ i.i.d. r.v. from a $U(0,1)$ distribution or from a $U_{\{1,2,\ldots,n\}}$
- transform the r.v´s to obtain more complex distributions

1. Introduction

There are fundamentally two different classes of methods for generate random numbers (RN's):

- Random numbers generated using something physical phenomenon of nature random, which can be slow and expensive; classic examples: tossing a coin or dice; modern methods that use quantum effects, thermal noise in electrical circuits, the radioactive decay calendar, etc.
- Pseudo-random numbers (PRN's) generated by computer programs, which prove to be fast and reproducible, but always of a deterministic and with apparently random output.

In this course we will only consider the PRN's.

Pseudo random numbers generator
A PRN generator is an algorithm that outputs sequences of numbers that can be used in place of an iid succession of true RN's.

Having recognized that a sequence of PRN's is cyclic, Lehmer suggested in 1949 to use number theory to try to make this period as long as possible. Today, almost all PRN generators are based on this idea.

1. Introduction

The PRN's used in software packages like R are more sophisticated than the linear congruent method presented, but share the following features (in R see help for .Random.seed):

- the sequence of PRN's generated depends on the seed X_0: different sequences obtained according to the seed; on the other hand, one obtains reproducible results by fixing the seed to a known value.
- Periodicity is a property shared by all generators computational. The period length is a measure of the quality of PRN's generators.
- another problem of generators concerns independence, since X_n deterministically depends on X_{n-1}. There are methods to assess and quantify this effect.

Finally, since PRN's are supposed to be a replacement for successions of iid RN's, a usual way of testing PRN's is to apply tests uniformity and independence statistics to the generated samples. Currently, used generators pass these tests

Objective: Generate independently the random variables $X_1, \ldots, X_n$ (one sample) with distribution function $F_X(x) = P(X_i \leq x)$ and probability function (discrete) or probability density function (continuous) $f_X(x)$.

It is assumed available the generation of independent random variables $U_1, \ldots, U_n$ with uniform distribution, $U_{[0;1]}$:

$$f_U(u) = \begin{cases} 0\ , & u \leq 0 \\ 1\ , & 0 \leq u \leq 1 \\ 0\ , & u \geq 1 \end{cases} \qquad F_U(u) = P(U \leq u) = \begin{cases} 0\ , & u \leq 0 \\ u\ , & 0 \leq u \leq 1 \\ 1\ , & u \geq 1 \end{cases}$$

(random numbers between 0 and 1) runif(n)

1. Introduction

The R generator command to PRN's is runif

The command

- u <- runif(n)
  produces n uniform random variables defined in $(0, 1)$
- u <- runif(n,a,b)
  produces n uniform random variables defined in (a, b)
- set.seed(439)
  fixes the seed, allowing reproducible results

generate 10^6 PRN's UNIFORMS(0,1)

```
n <- 10**6
```

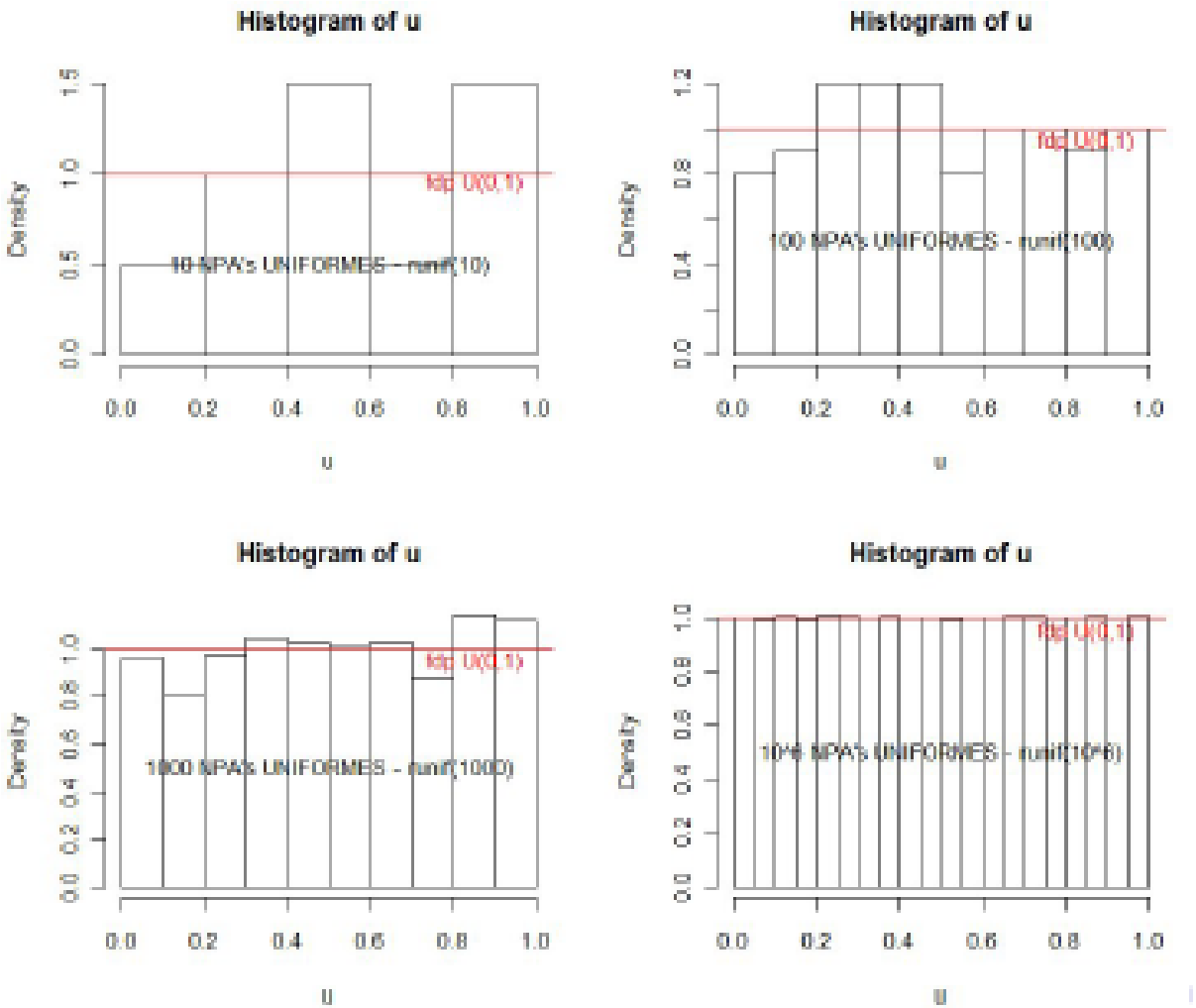
sample histogram of generated PRN's

```
hist(u, prob = TRUE)
```

p.d.f. of UNIFORM(0,1)

```
abline(h=1,col="red")
text(0.5,0.5,"10^6 PRN's UNIFORMES - runif(10^6)")
text(0.85,.95, "pdf U(0,1)",col="red")
```

Histogram vs U(0,1)pdf



2.1 Inverse transform method-univariate continuous variables

Inverse transform method:
Let X be a continuous r.v. with distribution function F_X.
Then $U = F_X(X)$ has uniform distribution $U(0,1)$.
Or, let U be a uniform random variable, $U \sim U(0,1)$. For any continuous distribution function F_X the random variable X defined by has distribution F_X.

Proof: (distribution function is a monotone increasing function)

$(\Rightarrow)$ $$F_U(u) = P(U \le u) = P(F_X(X) \le u) = P\left(X \le F_X^{-1}(u)\right) = F_X\left(F_X^{-1}(u)\right) = u$$

$(\Leftarrow)$ $$P(X \le x) = P\left(F_X^{-1}(U) \le x\right) = P(U \le F_X(x)) = F_X(x)$$

2.1 Inverse transform method-univariate continuous variables

Algorithm:

1) Generate U ~ $U_{[0;1]}$

2) Setting $X = F_X^{-1}(U)$, and X has distribution function F_X.

Uniform Distribution X ~ $U_{[a;b]}$ $\qquad f_X(x) = \frac{1}{b-a}, a \le x \le b$

$$F_X(x) = \int_a^x \frac{1}{b-a}\,dt = \frac{x-a}{b-a} \quad \text{for } a \le x \le b$$

$$u = F_X(x) \Leftrightarrow x = a + (b-a)u$$

Then it generates $U \sim U_{[0;1]}$ and it is taken

$$\boxed{X = a + (b-a)U}$$

2.1 Inverse transform method-univariate continuous variables

Exponencial Distribution X ~ Exp(λ)

$$f_X(x) = \lambda e^{-\lambda x},\ x \geq 0$$

$$F_X(x) = \int_0^x \lambda e^{-\lambda t} dt = 1 - e^{-\lambda x} \quad \text{for } x \geq 0$$

$$u = F_X(x) \Leftrightarrow u = 1 - e^{-\lambda x} \Leftrightarrow 1 - u = e^{-\lambda x} \Leftrightarrow x = -\frac{1}{\lambda}\ln(1-u)$$

And as $U \sim U_{[0;1]} \Rightarrow 1 - U \sim U_{[0;1]}$

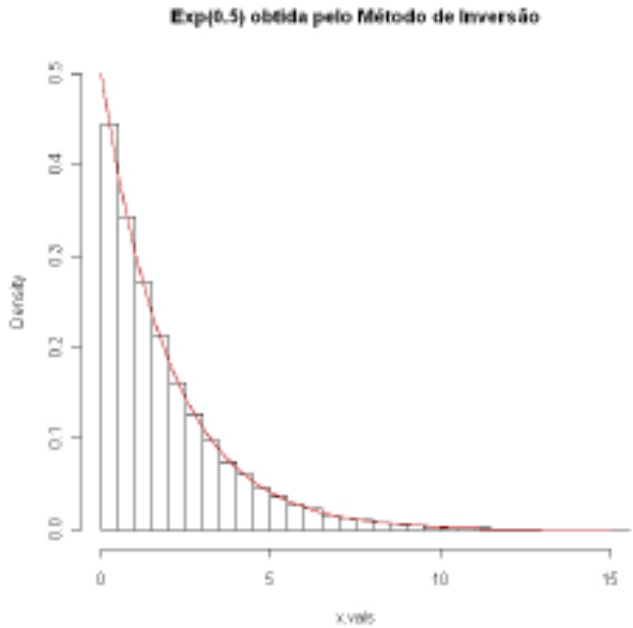
Then it generates $U \sim U_{[0;1]}$ and it is taken

$$\boxed{X = -\frac{1}{\lambda}\ln U}$$

2.1 Inverse transform method-univariate continuous variables

R: Inverse transform method → X ~ Exp(λ)

```
gera.exp <- function(npoints,lambda)
{
    u.vals <- runif(npoints)
    answer <- -1/lambda * log(u.vals)
    answer
}
```




```
# generates Exp(0.5) by Inverse Transform Method
lambda <- 0.5
n <- 1000
x.vals <- gera.exp(n,lambda)
```

```
hist(x.vals,breaks=50,freq=FALSE,xlim=c(0,15),ylim=c(0,0.5),main="Exp(λ) by
Inverse Transform Method")
```

```
xx.vals <- seq(0,15,0.001)
lines(xx.vals,dexp(xx.vals,lambda),col="red")
```

2.2 Inverse transform method-univariate discrete variables

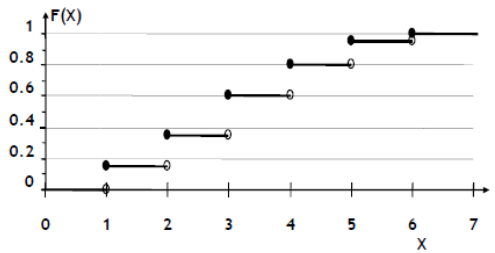
Let X be a discrete random variable with the discontinuity points of $F(x)$:

$$\ldots < x_{i-1} < x_i < x_{i+1} < \ldots$$

Inverse transform method:
$X = F_X^{-1}(U)$ has the distribution function F_X, if $U \sim U(0,1)$, where $F_X^{-1}(U) = min\{x : F_X(x) \geqslant u\}$.

Algorithm:
Obtain each value of distribution X

1. generate u from $U(0,1)$
2. get x_i if $F_X(x_{i-1}) < u \leqslant F_X(x_i)$



2.2 Inverse transform method-univariate discrete var.

Example: Let be X a random variable with probability function f_X and distribution function F_X given by

x	5	6	7
$f_X(x)$	0.2	0.5	0.3

$$F_X(x)=\begin{cases}0 & \text{if} & x<5\\ 0.2 & \text{if} & 5\le x<6\\ 0.7 & \text{if} & 6\le x<7\\ 1 & \text{if} & x\ge 7\end{cases}$$

Then, by inverse transform method

$$X=F_X^{-1}(U)=\begin{cases}5 & \text{if} & U\le F_X(5)\\ 6 & \text{if} & F_X(5)<U\le F_X(6)\\ 7 & \text{if} & F_X(6)<U\le F_X(7)\end{cases}=\begin{cases}5 & \text{if} & U\le 0.2\\ 6 & \text{if} & 0.2<U\le 0.7\\ 7 & \text{if} & U>0.7\end{cases}$$

Algorithm:

1) Generate U ~ $U_{[0;1]}$
2) If U ≤ 0.2 then X = 5
3) Else If U ≤ 0.7 then X = 6
4) Else X = 7

2.2 Inverse transform method-univariate discrete variable

Generalization: if X has probabilities $P(X=x_i)=p_i$, i=1,...,n

$$X = \begin{cases} x_1 & \text{if} \quad U \le p_1 \\ x_2 & \text{if} \quad p_1 < U \le p_1 + p_2 \\ \vdots & \\ x_i & \text{if} \quad \sum_{j=1}^{i-1} p_j < U \le \sum_{j=1}^{i} p_j \\ \vdots & \\ x_n & \text{if} \quad U > \sum_{j=1}^{n-1} p_j \end{cases}$$

$$P(X = x_i) = P\left(\sum_{j=1}^{i-1} p_j < U \le \sum_{j=1}^{i} p_j \right) = \sum_{j=1}^{i} p_j - \sum_{j=1}^{i-1} p_j = p_i$$

2.2 Inverse transform method-univariate discrete variable

Binomial distribution

X - number of successes on n independent experiences

p = P("success")

$X \sim Bi(n;p)$

Algorithm:

1) Initializing X = 0
2) For i = 1 to n do
3) Generate $U \sim U_{[0;1]}$
4) if $U \leq p$ then $X = X + 1$
5) Next i

2.2 Inverse transform method-univariate discrete variable

R: Binomial distribution → X ~ Bi(10;0.45)

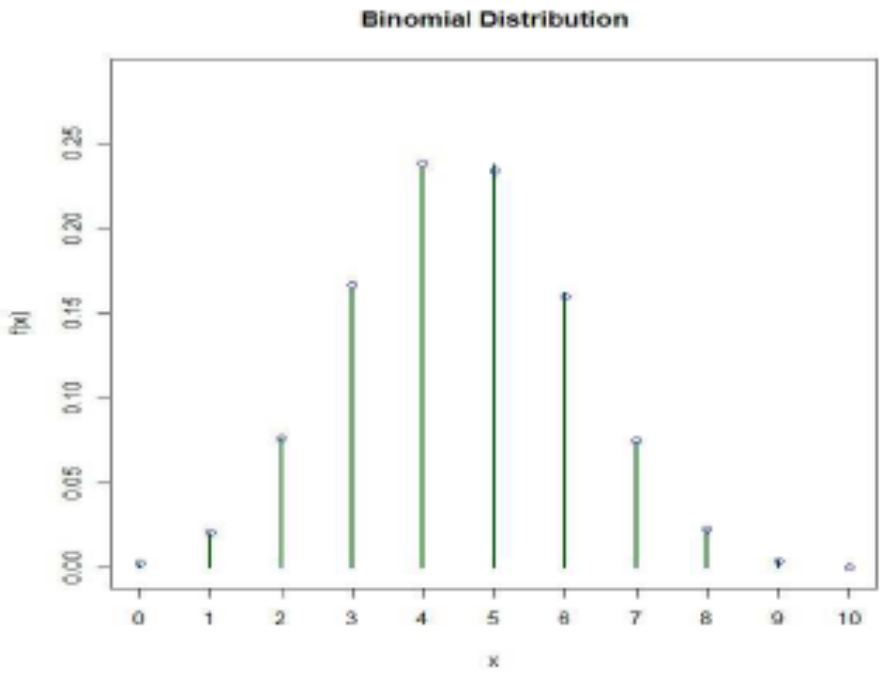
```
sim.binomial <- function(n,p,nvals)
{
    X.vals <- NULL
    for(j in 1:nvals) {
            x <- 0
            for(i in 1:n) {
                u <- runif(1)
                if(u <= p) {
                        x <- x + 1
                }
            }
            X.vals[j] <- x
    }
    X.vals
}

n <- 10
p <- 0.45
vals <- sim.binomial(n, p, 20000)


frel <- table(vals) / length(vals)
fmax <- max(frel) + 0.05

plot(frel, ylim=c(0,fmax), xlab="x", ylab="f(x)", main="Binomial Distribution", col="dark green")

x <- seq(0,10,1)
lines(x,dbinom(x,size=n,prob=p), col="blue", type="p", pch=21)
```



2.2 Inverse transform method-univariate discrete variable

Poisson distribution

$$X \sim Po(\lambda) \Leftrightarrow P(X=i) = \frac{e^{-\lambda}\lambda^i}{i!} = p_i \,, i = 0,1,2,...$$

Using inverse transform method,

$$X = i \quad \text{if} \quad \sum_{j=0}^{i-1} p_j < U \leq \sum_{j=0}^{i} p_j$$

$$p_{i+1} = \frac{e^{-\lambda}\lambda^{i+1}}{(i+1)!} = \frac{e^{-\lambda}\lambda^i}{i!}\frac{\lambda}{i+1} = \frac{\lambda}{i+1}p_i \quad \text{and} \quad p_0 = e^{-\lambda}$$

Algorithm:

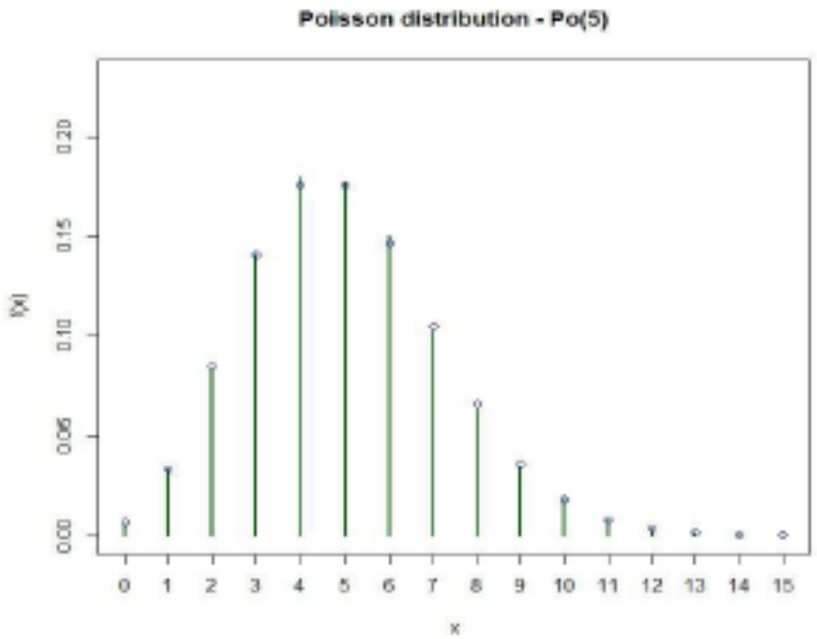
1) Initializing i = 0; $p = e^{-\lambda}$ and F = p
2) Generate U ~ $U_{[0;1]}$
3) While F ≤ U do
4) i <- i + 1
5) p = p * λ / i
6) F = F + p
7) X = i

2.2 Inverse transform method-univariate discrete variable

R: Poisson distribution → X ~ Po(5)

```
sim.poisson <- function(lambda,nvals)
{
    X.vals <- NULL
    for(j in 1:nvals) {
            u <- runif(1)
            i <- 0
            p <- exp(-lambda)
            F <- p
            while (F <= u) {
                i <- i + 1
                p = p * lambda/i
                F <- F + p
            }
            X.vals[j] <- i
    }
    X.vals
}

lambda <- 5
vals <- sim.poisson(lambda, 20000)
```




```
frel <- table(vals) / length(vals)
fmax <- max(dpois(trunc(lambda)+1,lambda),max(frel)) + 0.05

plot(frel, ylim=c(0,fmax), xlim=c(0,15), xlab="x", ylab="f(x)", main="Poisson distribution - Po(5)", col="dark
green")

x <- seq(0,max(vals),1)
lines(x,dpois(x,lambda), col="blue", type="p", pch=21)
```

2.2 Inverse transform method-univariate variable

R includes functions for the more usual distributions.

For each model exist 4 associated functions:

Model	**description**
rxxx	random generation
dxxx	density function / probability function
pxxx	cumulative probability function
qxxx	quantile function

For example, in normal model
rnorm $\hookrightarrow$ generator
dnorm $\hookrightarrow$ probability density function (or probability function)
pnorm $\hookrightarrow$ distribution function
qnorm $\hookrightarrow$ quantil function (inverse of df).

2.2 Inverse transform method-univariate variable

Some available functions in R

Models	
Uniform	runif dunif punif qunif
Normal	rnorm dnorm pnorm qnorm
t-Student	t rt dt pt qt
Exponential	rexp dexp pexp qexp
Log normal	rlnorm dlnorm plnorm qlnorm
Beta rbeta	dbeta pbeta qbeta
Binomial	rbinom dbinom pbinom qbinom
Poisson	rpois dpois ppois qpois

For more details use the command
> ??distribution

Multivariate distributions

Let be X=(X_1, …, X_n) a vector of random variables with joint distribution function F(.) and distribution marginal functions $F_{X_i}(.)$.

If X_1, …, X_n independent, then $$F(x_1,\cdots,x_n)=\prod_{i=1}^{n} F_{X_i}(x_i)$$

to generate X, simply independently generate each X_i.

Otherwise, the joint distribution function can be written by

$$F(x_1,\cdots,x_n)=F_{X_1}(x_1)\times F_{X_2|X_1}(x_2|X_1=x_1)\times\cdots\times F_{X_n|X_1,\ldots,X_{n-1}}(x_2|X_1=x_1,\cdots,X_{n-1}=x_{n-1})$$

where

$$F_{X_1}(x_1)=\int_{-\infty}^{x_1}\int_{-\infty}^{+\infty}\cdots\int_{-\infty}^{+\infty} f(t_1,\cdots,t_n)dt_1\cdots dt_n = F(x_1,\infty,\cdots,\infty)$$

$$F_{X_i|X_1,\ldots,X_{i-1}}(x_i|X_1=x_1,\cdots,X_{i-1}=x_{i-1})=\frac{\int_{-\infty}^{x_i}\int_{-\infty}^{+\infty}\cdots\int_{-\infty}^{+\infty} f(x_1,\cdots,x_{i-1},t_i,\cdots,t_n)dt_i\cdots dt_n}{f_{X_1,\cdots,X_{i-1}}(x_1,\cdots,x_{i-1})}$$

Multivariate distributions: algorithm

Algorithm to generate $X_1, \ldots, X_n$ not independent:

1) Generate $U_1,\ldots,U_n \sim U_{[0;1]}$

2) Solve the system
$$\begin{cases} U_1 = F_{X_1}(X_1) \\ U_2 = F_{X_2|X_1}(X_2|X_1) \\ \ldots \\ U_n = F_{X_n|X_1,\ldots,X_{n-1}}(X_2|X_1,\cdots,X_{n-1}) \end{cases}$$

in order to $X_1, \ldots, X_n$

Multivariate distributions: example

Let be (X,Y) a random vector with joint probability density function:

$$f(x,y)=\begin{cases}8xy & \text{if } 0\le y<x\le 1\\ 0 & \text{otherwise}\end{cases}$$

$$f_X(x)=\int_0^x 8xy\,dy=4x^3 \quad ,\text{if } 0\le x\le 1$$

$$F_X(x)=\int_0^x 4t^3\,dt=x^4 \quad ,\text{if } 0\le x\le 1$$

$$f_{Y|X}(y|X=x)=\frac{f(x,y)}{f_X(x)}=\frac{8xy}{4x^3}=\frac{2y}{x^2} \quad ,\text{if } 0\le y<x \text{ and where } 0<x\le 1$$

$$F_{Y|X}(y|X=x)=\int_o^y \frac{2t}{x^2}\,dt=\frac{y^2}{x^2} \quad ,\text{if } 0\le y<x \text{ and where } 0<x\le 1$$

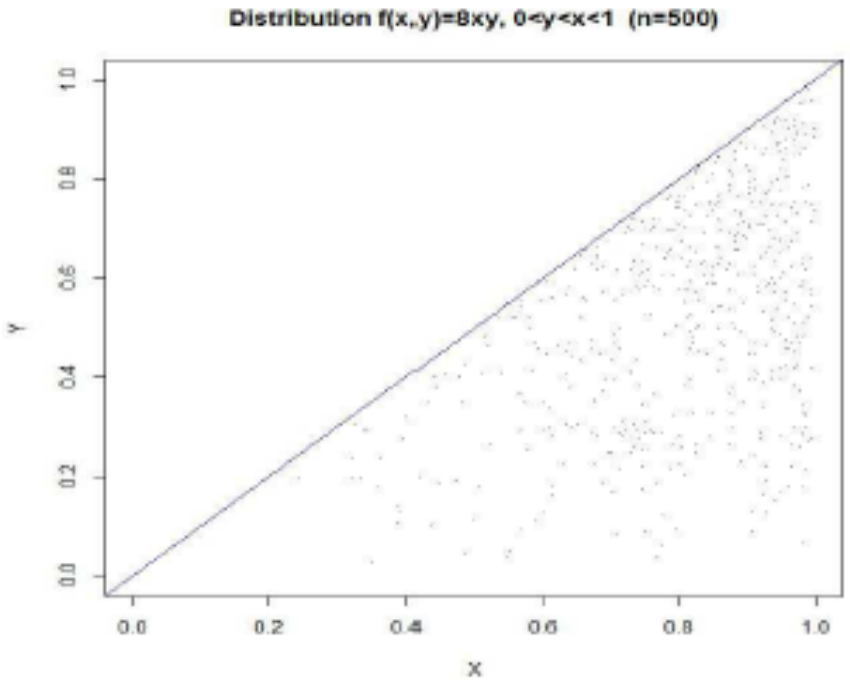
$$\begin{cases}U_1=F_X(X)\\ U_2=F_{Y|X}(Y|X)\end{cases}\Leftrightarrow\begin{cases}U_1=X^4\\ U_2=\dfrac{Y^2}{X^2}\end{cases}\Leftrightarrow\begin{cases}X=\sqrt[4]{U_1}\\ Y^2=X^2U_2\end{cases}\Leftrightarrow\begin{cases}X=\sqrt[4]{U_1}\\ Y=\sqrt[4]{U_1}\sqrt{U_2}\end{cases}$$

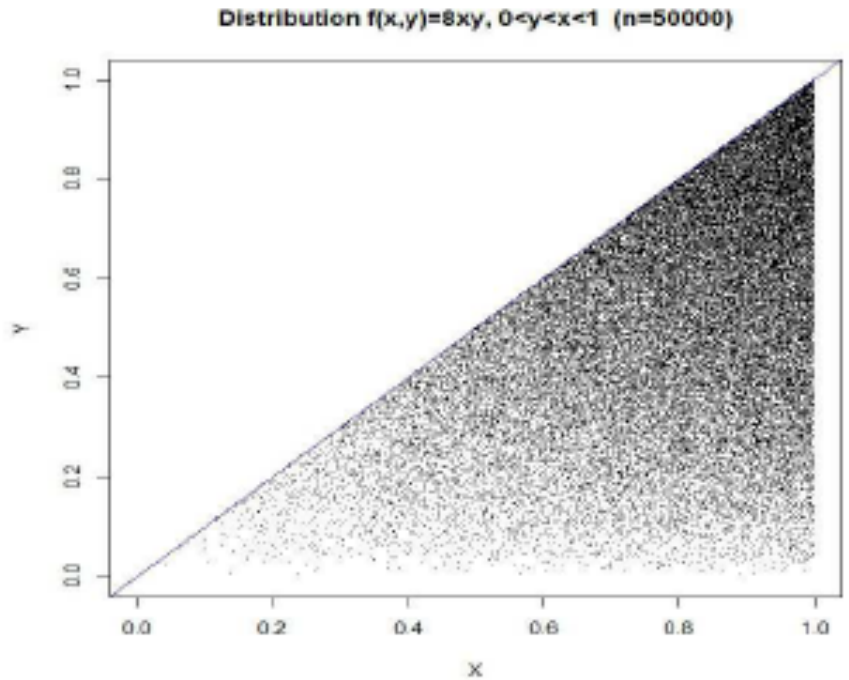
2.3 Inverse transform method-multivariate variable

R: Multivariate distribution f(x,y)

```
n <- 500
U1 <- runif(n)
U2 <- runif(n)

X <- U1^(1/4)
Y <- X * sqrt(U2)
XY <- cbind(X,Y)
plot(XY, pch=".", xlim=c(0,1), ylim=c(0,1), main="Distribution f(x,y)=8xy, 0<y<x<1  (n=500)")
abline(0,1,col="blue")
```





Continuous distributions: Rejection Method

Suppose we have a method for generating a random variable Y having density function g(.). We can use this as the basis for generating the random variable X having density function f(.).

Let k be a constant such that $\frac{f(x)}{g(x)} \leq k, \forall_x \Leftrightarrow f(x) \leq kg(x), \forall_x$

Efficient method if the functions f and g are similar (same means and similar domains).

Algorithm:

1. Generate y_i having density g(.)
2. Generate a random number u with U ~ $U_{[0;kg(y_i)]}$
3. if $u \leq f(y_i)$, then accept y_i as value of X else return to step 1

3. Rejection method

Or, alternatively
since

$$\frac{f(t)}{g(t)} \leqslant k, \forall t : f(t) > 0$$

to generate values from X with pdf f, it could be used equivalently

Algorithm: rejection method
Input: k

For each generated value x,

1. generate y from pdf g
2. generate u from $U(0,1)$
3. if $u < f(y)/(kg(y))$ accept y;
   otherwise, reject y and repeat.

Output: $x := y$

Important remarks

Consider
$N \hookrightarrow$ number of necessary generations of Y to get 1 generated value of X

- $N \sim Geom(\text{acceptance probability } 1/k) \rightarrow E(N) = k$
- N is called **rejection constant**
- k must be chosen such as $k = sup\frac{f(x)}{g(x)}$

3. Rejection method-example: Beta distribution

Want to simulate points of distribution Beta (2, 3):

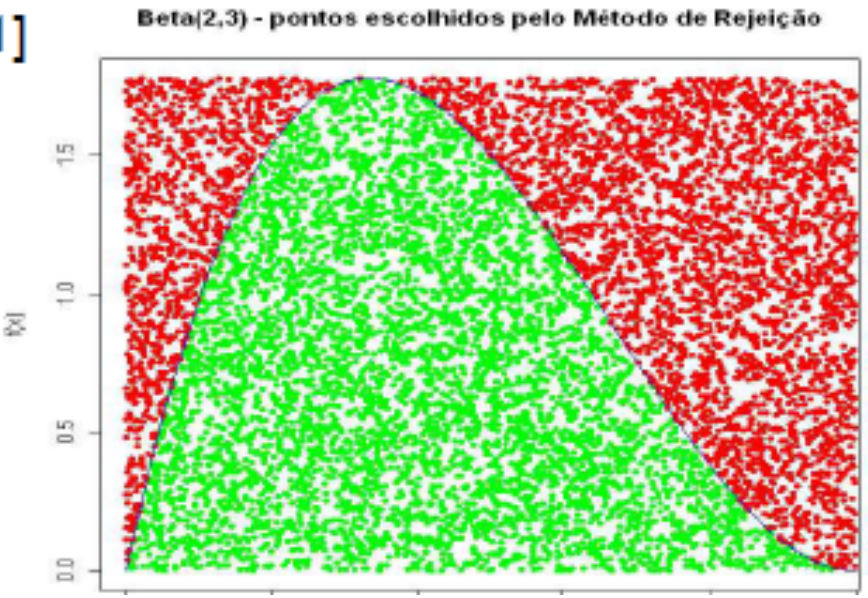
$$f(x) = 12x(1-x)^2 \quad , \text{for } 0 \le x \le 1$$

Using kg(x) = 16/9 (envelope) $\quad g(x) = 1 \quad , \text{for } 0 \le x \le 1$

$$f(x) \le \frac{16}{9}, \forall_x$$

Algorithm:

1. Generate y_i having uniform [0;1]
2. Generate u with U ~ $U_{[0;16/9]}$
3. if $u \le f(y_i)$,
   then accept yi as value of X
   else return to step 1



3. Rejection method-example: Beta distribution

R: Rejection Method → X ~ Beta(2,3)

```
reject.beta <- function(nvals)
{
    x.aceite <- NULL
    u.aceite <- NULL
    nx <- 0
    x.rejected <- NULL
    u.rejected <- NULL
    nrejected <- 0
    while(nx < nvals) {
            Y <- runif(1)
            U <- runif(1,0,16/9)
            if(U <= 12*Y*(1-Y)^2) {
                nx <- nx + 1
                x.aceite[nx] <- Y
                u.aceite[nx] <- U
            }
            else {
                nrejected <- nrejected + 1
                x.rejected[nrejected] <- Y
                u.rejected[nrejected] <- U
            }
    }
#plot(x.aceite,u.aceite,pch=20,col="green",xlab="x",ylab="f(x)",main="Beta(2,3) - pontos escolhidos pelo
Método de Rejeição")
#lines(x.rejected,u.rejected,pch=20,col="red",type="p")
    x.aceite
}

XX <- reject.beta(5000)
hist(XX,breaks=20,prob=TRUE,xlim=c(0,1),ylim=c(0,2),main="Beta(2,3) obtida pelo Método de Rejeição")
x.val <- seq(0,1,0.001)
lines(x.val,dbeta(x.val,2,3),col="blue")
```



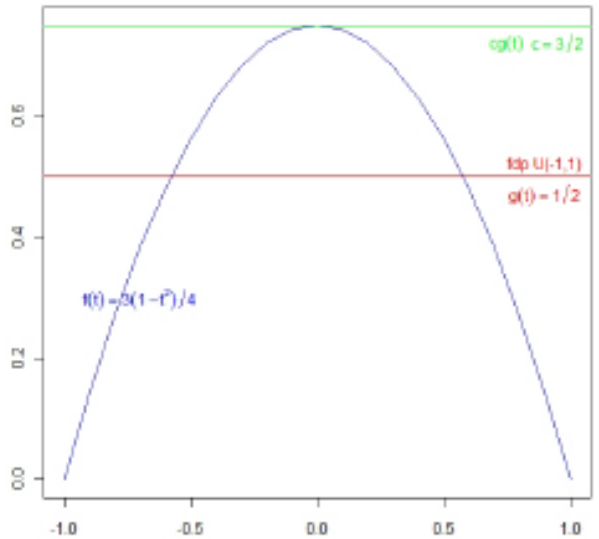
3. Rejection Method-example 2

Example:
Generate PRN's from the kernel density Epanechnikov given by

$$f(x) = \frac{3}{4}(1 - x^2), |x| \leqslant 1$$

Consider the auxiliar model for Y given by $U(-1, 1)$ distribution;

then $k = 3/2$



3. Rejection method-example Epanechnikov

In the general algorithm, we get

$$f(y)/(kg(y)) = \frac{3}{4}(1-y^2)/(3/4) = 1-y^2$$

Algorithm:PRN Epanechnikov

Repeat,

generate $y \sim U(-1,1)$ and $u \sim U(0,1)$, independently until $u < 1-y^2$

Output: $x := y$

It is necessary to generate in mean 1.5 u values from $U(0,1)$ distribution in order to get 1 observation of the target distribution

Exercise

Implement in R a function that allows you to generate a sample of dimension n of rv's with pdf f(x)

3. Rejection method-truncated model

It is often useful to work with a classic model, but truncated in the tails, either from the right or from the left, or only on one side.

Definition of truncated variable

Let X be a rv with df F and pdf f. The r.v defined by

$$X|a \leqslant X \leqslant b$$

is called left truncated of a and right truncated of b; it has pdf

$$\frac{f(x)}{F(b)-F(a)}, x \in (a,b)$$

Remark: In the previous definition, can be taken $a = -\infty$ or $b = +\infty$, producing models truncated to the right of b or to the left of a, respectively.

3. Rejection method-truncated model

It is often useful to work with a classic model, but truncated in the tails, either from the right or from the left, or only on one side.

Algorithm: truncated variable

Repeat

generate x with df F

until $x \in (a, b)$

output: x

Remark: The mean number of values generated necessary to get one truncated observation is $\frac{1}{F(b)-F(a)}$

This is a very useful procedure when it is pretended to cut just the extreme points of the original model.

4. Sums of random variables

A very important notion is concerning **Sums of random variables**

Sums of vandom variables

Let $X_1, \ldots, X_n$ be independent random variables with df $F_1, \ldots, F_n$ respectively; the variable sum is defined as

$$\sum_{i=1}^{n} X_i$$

whose correspondent distribution is called convolution product.

Remark: The distribution of the variables can be the same or different, the sum of the coefficients can be any value.

Exercise

Generate n values from

$$X := \frac{1}{3}X_1 + \frac{2}{3}X_2$$

with $X_1 \sim N(0,1)$ and $X_2 \sim N(3,1)$, independents.

Algorithm: PRN from $X := \frac{1}{3}X_1 + \frac{2}{3}X_2$

1. Generate x_1 from $N(0,1)$
2. Generate x_2 from $N(3,1)$
3. Output: $X := \frac{1}{3}x_1 + \frac{2}{3}x_2$

If it is pretended to get a sample with n generated values, repeat the procedure n times.

4. Mixtures of random variables

A very important notion is concerning **Mixture model** based on the composition method.

Mixture Model

- A rv X is a discrete model mixture if

$$F_X(x) = \sum_{i=1}^{k} \theta_i F_i(x), \theta_i > 0, \sum_{i=1}^{k} \theta_i = 1$$

  θ_i are called **weights** or **mixture probabilities**.

- A rv X is a continuous model mixture if

$$F_X(x) = \int_{-\infty}^{+\infty} f_Y(y) F_{X|Y=y}(x) dy, f_Y(y) > 0, \int_{-\infty}^{+\infty} f_Y(y) dy = 1$$

  $f(y)$ is called **mixture function**.

Composition Method

To generate values of a mixture distribution.
A random variable X has a mixture distribution if its probability density function is of the type $f_X(x) = \sum_{i=1}^{r} p_i f_i(x)$

where, for each i, $f_i(x)$ is a probability density function and $\{p_1, p_2, \ldots, p_r\}$ is a set of weights satisfying $\sum_{i=1}^{r} p_i = 1$.
p_i - probability of one observation come from distribution with probability density function $f_i(x)$.

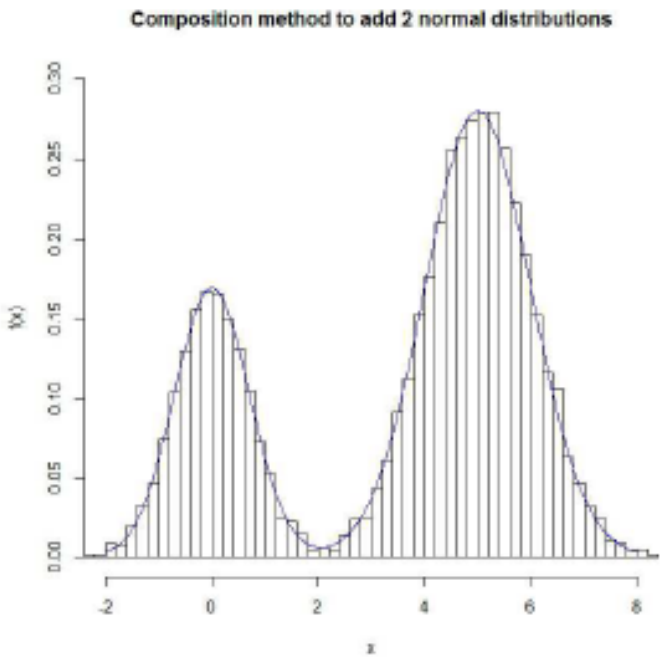
Algorithm:

1. Select index j of {1, ..., r} according to the probabilities $\{p_1, \ldots, p_r\}$
2. Generate X_j having density $f_j(x)$
3. Set $X = X_j$

R: Composition Method → N(0;0.5) and N(5;1)

Simulation of a mixture of two normals N(0;0.5) and N(5;1) with probabilities $p_1 = 0.3$ and $p_2 = 0.7$.

```
sim.fn3 <- function(p1,m1,m2,v1,v2,nvals)
{
    X.vals <- NULL
    nx <- 0
    X <- polar.normal(nvals)
    while(nx < nvals) {
            p <- runif(1)
            nx <- nx + 1
            if(p < p1) {
                X.vals[nx] <- m1 + X[nx] * sqrt(v1)
            }
            else {
                X.vals[nx] <- m2 + X[nx] * sqrt(v2)
            }
    }
    X.vals
}
```




```
vals <- sim.fn3(0.3,0,5,0.5,1,10000)

hist(vals,prob=TRUE,nclass=50, xlim=c(-2,8), ylim=c(0,0.30),xlab="x",ylab="f(x)",main="Composition method to
add 2 normal distributions")
x <- seq(-2,8,0.01)
lines(x, 0.3 * dnorm(x,0,sqrt(0.5)) + 0.7 * dnorm(x,5,1),col="blue")
```

Exercise

Generate n values from the model

$$f_X(x) := \frac{1}{3} f_{X_1}(x) + \frac{2}{3} f_{X_2}(x)$$

with $X_1 \sim N(0,1)$ and $X_2 \sim N(3,1)$, independents.

Algorithm: PRN from $f_X(x) := \frac{1}{3} f_{X_1}(x) + \frac{2}{3} f_{X_2}(x)$

1. Generate $k \in \{1,2\}$, such as $p(1) = \frac{1}{3}, p(2) = \frac{2}{3}$
2. If $k = 1$, generate x_1 from $N(0,1)$
   if $k = 2$, generate x_2 from $N(3,1)$

If it is pretended to get a sample with n generated values, repeat the procedure n times.

Graphical fit method - QQ plot

Although subjective, this method allows a rapid visual confirmation of quality adjustment of a particular model to the data.

Build the quantiles graphic of the collected data, that is, $x_{(1)}, \ldots, x_{(n)}$ versus corresponding quantiles of the distribution: $F^{-1}\left(\frac{1-0.5}{n}\right), \cdots, F^{-1}\left(\frac{n-0.5}{n}\right)$

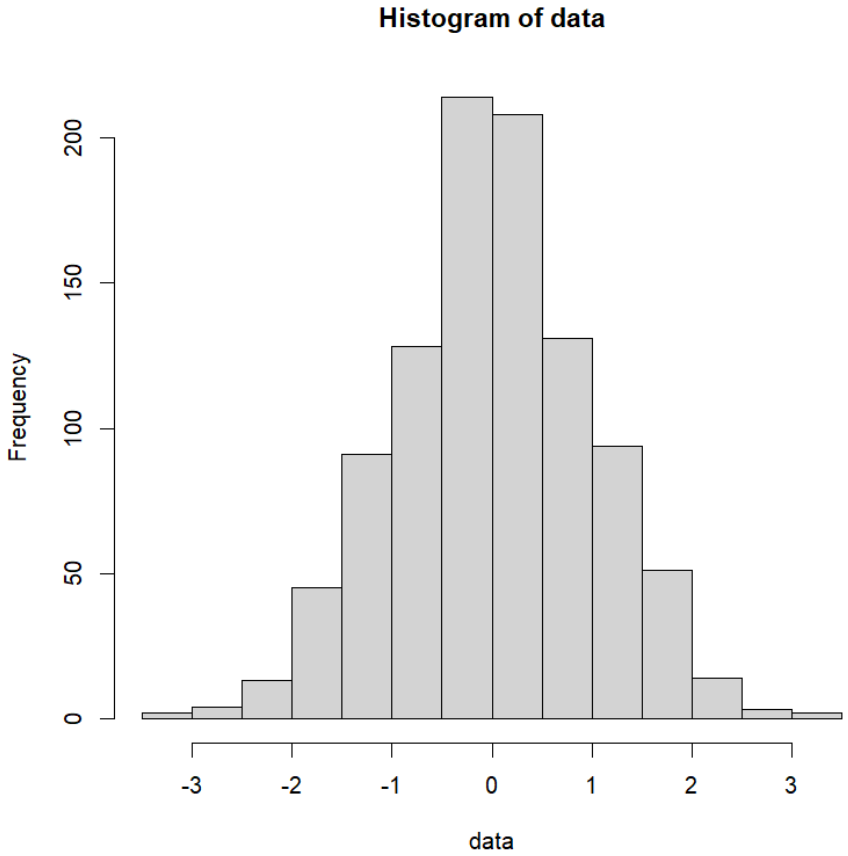
- If the distribution is adequate adjusted, the points on the graph are distributed randomly close to a straight line.
- If the points were significantly deviate from a straight line then the model is not adequate.

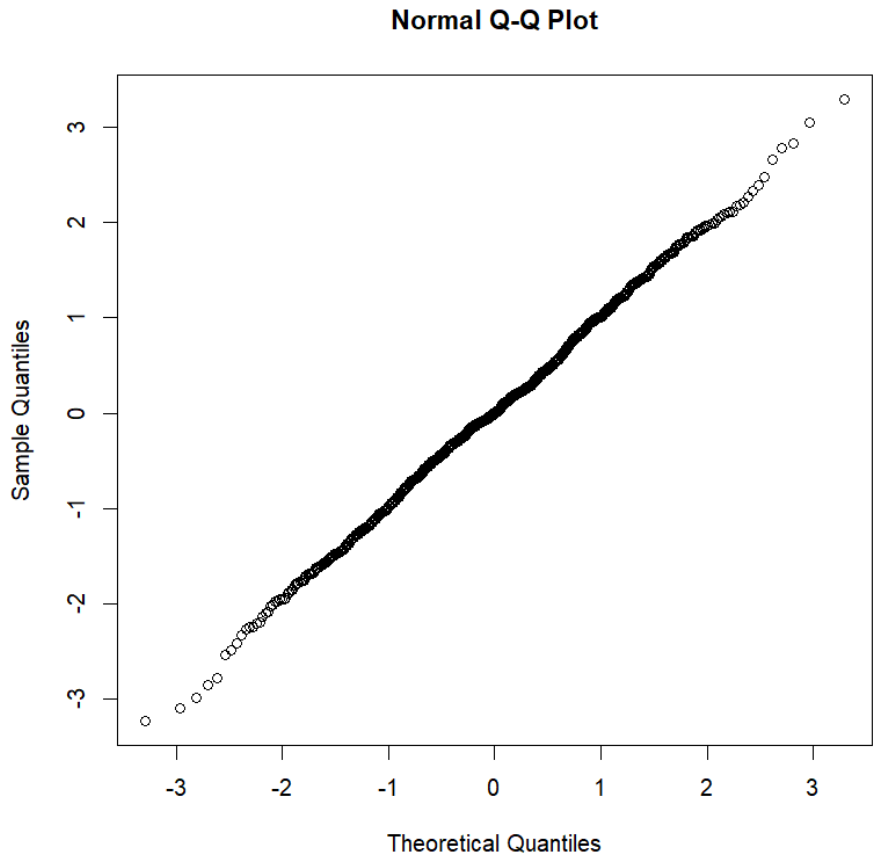
It can be also possible to compare the distributions of 2 samples in the graph of quantiles, by drawing the quantiles $x_{(1)}, \ldots, x_{(n)}$ versus the other quantiles $y_{(1)}, \ldots, y_{(n)}$. If they have the same distribution then these points are distributed randomly close to a straight line.

5. Adequacy of the statistical model

Example: Simulate 1000 observations from N(0,1), look at the histogram and qqplot

```
> data=rnorm(1000)
> hist(data)
> qqnorm(data)
```





References - Bibliography

- Fishman, G. S.(1996). Monte Carlo. Concepts, Algorithms and Applications. Springer.
- Lehmer, D. H. (1949). "Mathematical methods in large-scale computing units". Proceedings of a Second Symposium on Large-Scale Digital Calculating Machinery. pp. 141–146. MR 0044899. (journal version: Annals of the Computation Laboratory of Harvard University, Vol. 26 (1951)).
- Ross, S. M.(1997). Simulation. Academic Press.